

TEMA 2 – HOJA DE EJERCICIOS VII

Se plantean diferentes ejercicios sobre ontologías y su representación visual por medio de grafos.

Las ontologías son un tipo de representación estructurada del conocimiento donde los conceptos se organizan y relacionan entre sí. A menudo, las ontologías se representan visualmente por medio de grafos, donde cada nodo corresponde a un concepto y las aristas representan las relaciones existentes entre dichos conceptos.

1. WordNet es un recurso léxico que agrupa palabras en conjuntos de *synsets*, definiendo relaciones semánticas y jerárquicas entre ellos. Aunque no presenta todas las características formales de una ontología léxica en sentido estricto, comparte numerosas características fundamentales con ellas.

Se quiere visualizar gráficamente la relación entre el primer *synset* de la palabra “dog” (dog.n.01) y sus correspondientes hipónimos. Para ello, se usarán las librerías NetworkX y Matplotlib:

- NetworkX es una biblioteca de Python para el estudio de grafos y análisis de redes.
- Matplotlib es una librería Python open source que permite crear visualizaciones de datos.

A continuación, se proporciona un código de ejemplo que muestra un grafo en el que hay 4 nodos (n1, n2, n3 y n4) y relaciones entre ellos (n1 -r1-> n2, n2 -r2-> n3, n4 -r3-> n3):

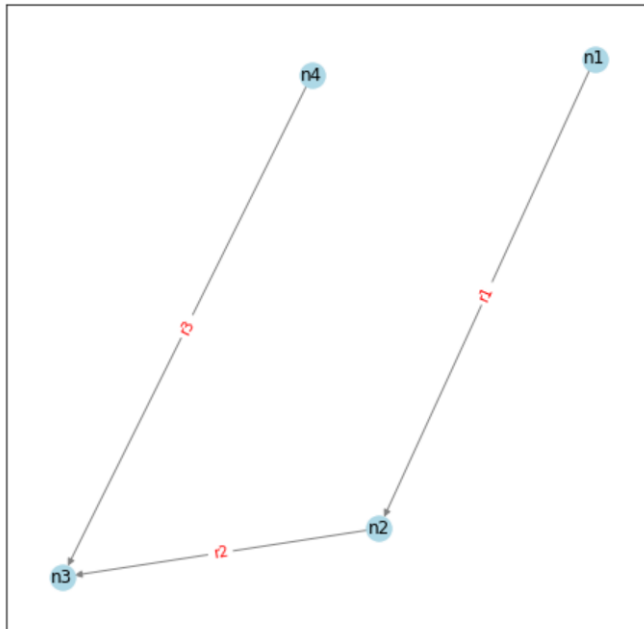
```
from nltk.corpus import wordnet as wn
import networkx as nx
import matplotlib.pyplot as plt

G = nx.DiGraph()
G.add_edge('n1', 'n2', label='r1')
G.add_edge('n2', 'n3', label='r2')
G.add_edge('n4', 'n3', label='r3')

plt.figure(figsize=(8, 8))

pos = nx.spring_layout(G, k=0.5)
nx.draw_networkx(G, pos, node_color='lightblue', edge_color='gray')
edge_labels = nx.get_edge_attributes(G, 'label')
nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels, font_color='red', font_size=10)
plt.show()
```

La visualización del grafo se muestra a continuación:



2. En WordNet, las relaciones de hiperonimia e hiponimia no están limitadas a un solo nivel jerárquico. Esto implica que, a partir de un concepto de partida, no solo es posible identificar sus relaciones directas, sino también las relaciones a niveles sucesivos. Es decir, además de poder obtener los hiperónimos de un concepto (como dog.n.01), podemos seguir explorando y obtener los hipónimos de sus propios hipónimos.

En este ejercicio, se desea recorrer recursivamente y visualizar con NetworkX todos los hiperónimos a partir del primer *synset* de la palabra “dog” (dog.n.01), mostrando una jerarquía completa de conceptos ascendentes. Reflexionar sobre la utilidad que estas relaciones y jerarquías pueden ofrecer en diversos casos de uso de PLN.

Para mejorar la visualización del grafo, se recomienda cambiar el layout al siguiente:

```
pos = nx.kamada_kawai_layout(G)
```

3. Las tripletas SVO (Sujeto-Verbo-Objeto) son una forma de fragmentar una frase en sus tres partes principales: Sujeto (entidad que realiza la acción), Verbo (la acción que se realiza) y Objeto (la entidad que recibe la acción). Estas tripletas proporcionan una manera sencilla de estructurar las diferentes frases que componen los documentos de un corpus, pudiendo ser un punto de partida para la creación de relaciones y grafos semánticos.

- a. Se desean extraer las tripletas SVO del siguiente conjunto de frases:

- Jack eats pizza.
- Jack reads a book.
- Alice sells pizza.
- Bob writes a book.

Para ello, se recomienda usar textacy, librería construida sobre spaCy que extiende sus funcionalidades. En particular, se recomienda emplear la función `textacy.extract.subject_verb_object_triples(doc)` de dicha librería, la cual recibe como entrada un objeto *Doc* de spaCy.

Nota: [textacy](#)

- b. Una vez extraídas las anteriores relaciones semánticas, se desea visualizarlas con NetworkX. Reflexionar sobre su utilidad para estructurar información anteriormente no estructurada.
4. Utilizando la librería [Owlready2](#) que permite cargar ontologías y manejarlas como si fueran objetos en Python, se va a consultar información de una [ontología de prueba sobre pizzas](#).

En primer lugar, hay que cargar la ontología, para ello hay que proporcionar el IRI donde se encuentra la ontología. Los IRI se utilizan como identificadores para las ontologías y son como URLs:

```
onto = get_ontology("http://www.lesfleursdunormal.fr/static/_downloads/pizza_onto.owl").load()
```

Consultar aquí, en la sección “Accessing the content of an ontology” los diferentes métodos que se podrían aplicar al objeto anterior onto para consultar el contenido de una ontología.

Se pide lo siguiente:

- a. Mostrar el grafo de la ontología. Probar el código que se proporciona (PizzaOntologyGraph.py)
- b. Mostrar las clases de la ontología.
- c. Mostrar los individuos (las instancias concretas) de la ontología.
- d. Se quieren hacer búsquedas sobre la ontología (consultar [aquí](#) ejemplos):
 - a. Buscar todas las entidades o conceptos que tenga al final del IRI “Pizza”.
 - b. Buscar todos los individuos que tengan alguna relación ‘has_topping’ con otros individuos.